

BUGFIX: Sistema de Detecção de Progresso

Data: 2026-03-10

Branch: ModularizacaoSistema

Problema Relatado

O sistema estava cancelando tarefas após 6 ciclos “sem progresso”, mesmo quando a tarefa foi realmente concluída. Isso indicava falsos negativos na detecção de progresso.

Sintomas:

- Tarefas canceladas com mensagem “6 turnos sem progresso”
 - Porém, ao verificar, a tarefa havia sido executada corretamente
 - O sistema não estava reconhecendo o progresso real feito pelo agente
-

Bugs Encontrados e Corrigidos

Bug #1: `computeTurnProgress` só analisava o primeiro comando

Localização: `taskExecutionService.js` (função `computeTurnProgress`)

Problema: O código original só inferia arquivos lidos do PRIMEIRO comando executado:

```
// ANTES (BUG)
const inferred = toolResult.commandsExecuted.length > 0
  ? inferReadOnlyEvidenceFromCommand(toolResult.commandsExecuted[0], cwd)
  : { filesRead: [], modifiedFiles: [] };
```

Correção: Agora itera por TODOS os comandos executados:

```
// DEPOIS (CORRIGIDO)
for (const command of commandsExecuted) {
  const inferred = inferReadOnlyEvidenceFromCommand(command, cwd);
  allInferredReads = mergeUniquePaths(allInferredReads, inferred.filesRead || []);
}
```

Bug #2: `computeTurnProgress` não considerava `filesWritten`

Localização: `taskExecutionService.js` e `evidenceService.js`

Problema: Apenas `modifiedFiles` era considerado para mutações:

```
// ANTES (BUG)
const meaningfulMutations = uniquePaths(
  toolResult.modifiedFiles || [],
).filter((file) => !isEphemeralArtifact(file));
```

Correção: Agora considera TANTO `modifiedFiles` QUANTO `filesWritten` :

```
// DEPOIS (CORRIGIDO)
const allMutations = mergeUniquePaths(
  toolResult.modifiedFiles || [],
  toolResult.filesWritten || []
);
const meaningfulMutations = allMutations.filter((file) => !isEphemeralArtifact(file));
```

Bug #3: `touchedFiles` não entrava no cálculo de progresso

Localização: `taskExecutionService.js` e `evidenceService.js`

Problema: Arquivos “tocados” (lidos mas não modificados) não eram considerados como progresso:

```
// ANTES (BUG)
hasMeaningfulProgress:
  meaningfulReads.length > 0 ||
  meaningfulMutations.length > 0
```

Correção: Agora `touchedFiles` também conta como progresso (especialmente útil para tarefas de análise):

```
// DEPOIS (CORRIGIDO)
hasMeaningfulProgress:
  meaningfulReads.length > 0 ||
  meaningfulMutations.length > 0 ||
  touchedFiles.length > 0 // NOVO!
```

Bug #4: Regex de `isEphemeralArtifact` muito restritivo

Localização: `taskExecutionService.js` e `evidenceService.js`

Problema: O regex `/^terminal-\d+\.log$/i` só capturava IDs numéricos:

```
// ANTES (BUG)
/^terminal-\d+\.log$/i.test(base) ||
/^relatorio-\d+\.txt$/i.test(base)
```

Correção: Regex mais flexível para capturar qualquer formato de ID:

```
// DEPOIS (CORRIGIDO)
/^terminal-[^/]+\log$/i      // Qualquer ID
/^relatorio-[^/]+\txt$/i     // Qualquer ID
```

Logs de Debug Adicionados

Prefixo: [PROGRESS-DEBUG]

Os logs podem ser filtrados usando:

```
node monitor.js 2>&1 | grep "\[PROGRESS-DEBUG\]"
```

Logs Disponíveis

Início de cada turno

```
[PROGRESS-DEBUG]
[PROGRESS-DEBUG] INÍCIO DO TURNO 1
[PROGRESS-DEBUG]
[PROGRESS-DEBUG] turnosSemProgresso atual: 0/6
```

toolResult recebido

```
[PROGRESS-DEBUG] --- toolResult recebido ---
[PROGRESS-DEBUG] toolCall.name: read
[PROGRESS-DEBUG] toolResult.success: true
[PROGRESS-DEBUG] toolResult.filesRead: ["/path/to/file.js"]
[PROGRESS-DEBUG] toolResult.filesWritten: []
[PROGRESS-DEBUG] toolResult.modifiedFiles: []
[PROGRESS-DEBUG] toolResult.touchedFiles: ["/path/to/file.js"]
[PROGRESS-DEBUG] toolResult.commandsExecuted: []
[PROGRESS-DEBUG] toolResult.executionDiagnostics: {}
```

Cálculo de progresso detalhado

```
[PROGRESS-DEBUG] ===== INÍCIO computeTurnProgress =====
[PROGRESS-DEBUG] cwd: /home/ubuntu/projeto
[PROGRESS-DEBUG] contractFulfilled: false
[PROGRESS-DEBUG] Comandos executados: 1
[PROGRESS-DEBUG] [0] find . -name "*.js"...
[PROGRESS-DEBUG] filesRead direto do toolResult: 0
[PROGRESS-DEBUG] Total de leituras (antes do filtro): 2
[PROGRESS-DEBUG] EXCLUÍDO (efêmero) da leitura: /path/terminal-123.log
[PROGRESS-DEBUG] meaningfulReads após filtro: 1
[PROGRESS-DEBUG] ☒ /home/ubuntu/projeto/src/index.js
```

Decisão final de progresso

```
[PROGRESS-DEBUG] ----- DECISÃO FINAL -----
[PROGRESS-DEBUG] hasReads: true
[PROGRESS-DEBUG] hasMutations: false
[PROGRESS-DEBUG] hasTouched: true
[PROGRESS-DEBUG] noOpMutation: false
[PROGRESS-DEBUG] contractFulfilled: false
[PROGRESS-DEBUG] >>> hasMeaningfulProgress: true <<<
[PROGRESS-DEBUG] ===== FIM computeTurnProgress =====
```

Atualização do contador

```
[PROGRESS-DEBUG]  PROGRESSO DETECTADO! Resetando contador.
[PROGRESS-DEBUG] turnosSemProgresso: 2 → 0
```

ou

```
[PROGRESS-DEBUG]  SEM PROGRESSO! Incrementando contador.
[PROGRESS-DEBUG] turnosSemProgresso: 2  3
[PROGRESS-DEBUG] --- ANÁLISE: Por que não houve progresso? ---
[PROGRESS-DEBUG]  Nenhum arquivo significativo foi lido
[PROGRESS-DEBUG]  Nenhum arquivo significativo foi modificado
[PROGRESS-DEBUG]  Nenhuma tool call foi detectada na resposta do agente
```

Como Interpretar os Logs

Cenário 1: Progresso detectado corretamente

```
[PROGRESS-DEBUG] meaningfulReads após filtro: 3
[PROGRESS-DEBUG] >>> hasMeaningfulProgress: true <<<
[PROGRESS-DEBUG]  PROGRESSO DETECTADO! Resetando contador.
```

 O sistema detectou que o agente leu 3 arquivos significativos.

Cenário 2: Falso negativo (agora corrigido)

ANTES (bug):

```
[PROGRESS-DEBUG] filesWritten do toolResult: ["/path/newfile.js"]
[PROGRESS-DEBUG] modifiedFiles do toolResult: []
[PROGRESS-DEBUG] meaningfulMutations após filtro: 0 # BUG! Deveria ser 1
[PROGRESS-DEBUG] >>> hasMeaningfulProgress: false <<< # ERRADO!
```

DEPOIS (corrigido):

```
[PROGRESS-DEBUG] filesWritten do toolResult: ["/path/newfile.js"]
[PROGRESS-DEBUG] modifiedFiles do toolResult: []
[PROGRESS-DEBUG] Total de mutações (antes do filtro): 1 # Agora considera filesWritten
[PROGRESS-DEBUG] meaningfulMutations após filtro: 1
[PROGRESS-DEBUG] >>> hasMeaningfulProgress: true <<< # CORRETO!
```

Cenário 3: Arquivo efêmero excluído corretamente

```
[PROGRESS-DEBUG] EXCLUÍDO (efêmero) da mutação: /tasks/terminal-123.log
[PROGRESS-DEBUG] meaningfulMutations após filtro: 0
```

✓ Arquivos de log do sistema não contam como progresso.

Arquivos Modificados

1. `src/services/taskExecutionService.js`
 - Função `computeTurnProgress()` - corrigida e com logs
 - Função `isEphemeralArtifact()` - regex melhorado
 - Loop principal de execução - logs extensivos
2. `src/services/evidenceService.js`
 - Função `computeTurnProgress()` - mesmas correções
 - Função `isEphemeralArtifact()` - regex melhorado

Critérios de Progresso (Atualizado)

Um turno é considerado “com progresso” se:

Condição	Progresso?
<code>contractFulfilled = true</code>	✓ SIM
Leu arquivo não-efêmero	✓ SIM
Modificou arquivo não-efêmero	✓ SIM
Tocou arquivo não-efêmero	✓ SIM (NOVO!)
Criou arquivo via <code>filesWritten</code>	✓ SIM (CORRIGIDO!)
<code>noOpMutation = true</code>	✗ NÃO
Apenas modificou <code>terminal-*.log</code>	✗ NÃO
Apenas modificou <code>relatorio-*.txt</code>	✗ NÃO

Arquivos Efêmeros (não contam como progresso)

- `terminal-*.log` - Logs do terminal
- `relatorio-*.txt` - Relatórios do sistema
- `*.lock` - Arquivos de lock
- `monitor-state.json` - Estado do monitor
- `.done` (oculto) - Arquivos done ocultos

NOTA: `done-123.done` NÃO é efêmero! Criar o arquivo `.done` é parte do objetivo da tarefa.

Testando as Correções

Para verificar se as correções estão funcionando:

```
# Rodar o monitor e filtrar logs de progresso
node monitor.js 2>&1 | grep "\[PROGRESS-DEBUG\]"

# Ou salvar em arquivo para análise
node monitor.js 2>&1 | tee monitor.log
grep "\[PROGRESS-DEBUG\]" monitor.log
```

Autor

Sistema corrigido em 2026-03-10 via análise detalhada do código.